

PROFESSIONAL CERTIFICATE IN MACHINE LEARNING AND ARTIFICIAL INTELLIGENCE

Office Hours with Ahmet F Cakmak

11/10, 2025 at 01:00 AM UTC time
9:00 PM US Eastern Time

**Overfitting - Underfitting and
Solutions**

AGENDA

1. Linear Regression
2. Polynomial Regression
3. Underfitting and Overfitting for Regression
4. Possible Solutions
5. Regularization and Grid Search (Decision Trees).

References:

Luis Serrano, [Grokking Machine Learning](#)

Luis Serrano's [GitHub Page](#) related to the book.

Ahmet Cakmak's independent study [GitHub Page](#).

Lagrange Multipliers from Khan Academy [video link](#).

OVERFITTING - UNDERFITTING

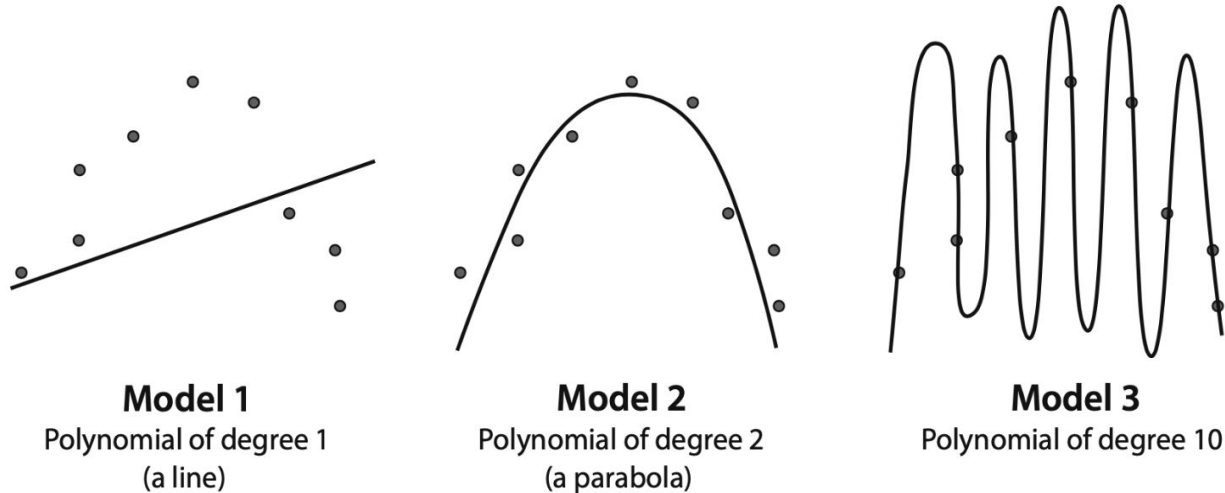


Figure 4.3 Fitting three models to the same dataset. Model 1 is a polynomial of degree 1, which is a line. Model 2 is a polynomial of degree 2, or a quadratic. Model 3 is a polynomial of degree 10. Which one looks like the best fit?

OVERFITTING - UNDERFITTING

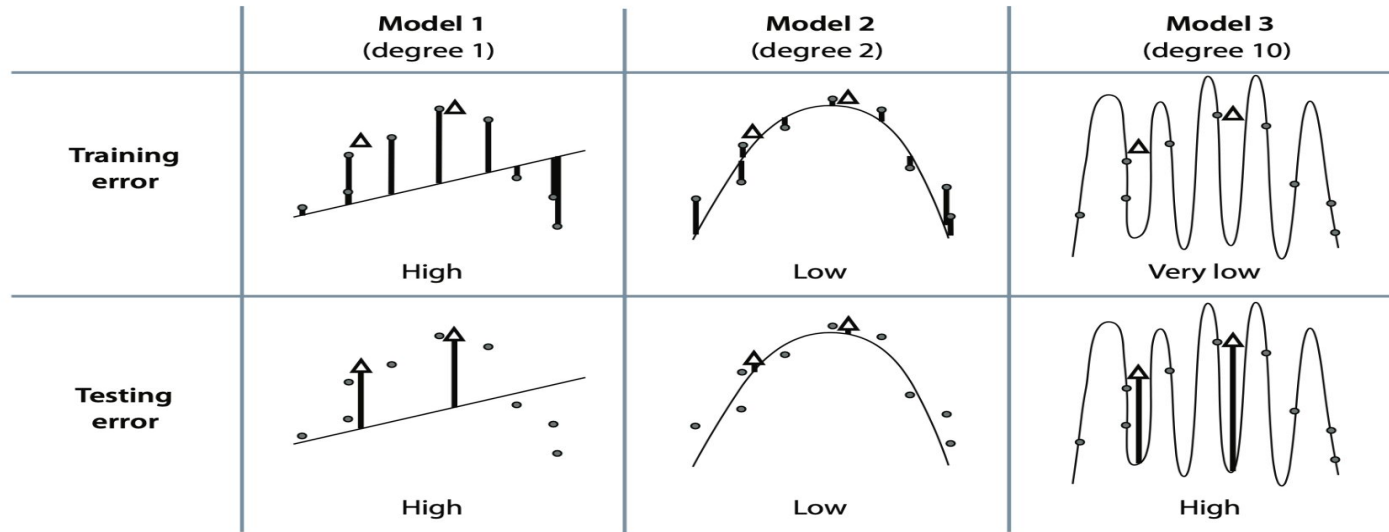


Figure 4.4 We can use this table to decide how complex we want our model. The columns represent the three models of degree 1, 2, and 10. The rows represent the training and the testing error. The solid circles are the training set, and the white triangles are the testing set. The errors at each point can be seen as the vertical lines from the point to the curve. The error of each model is the mean absolute error given by the average of these vertical lengths. Notice that the training error goes down as the complexity of the model increases. However, the testing error goes down and then back up as the complexity increases. From this table, we conclude that out of these three models, the best one is model 2, because it gives us a low testing error.

OVERFITTING - UNDERFITTING

Model Performance: Underfitting, Proper Fit, and Overfitting

Models can exhibit three primary behaviors:

1. **Underfitting:** The model is too simple to capture the patterns in the dataset.
2. **Good Fit:** The model has the appropriate level of complexity to represent the dataset effectively.
3. **Overfitting:** The model is excessively complex, capturing noise or irrelevant details in the dataset.

Performance in the Training Set:

- An **underfit model** performs poorly, resulting in a high training error because it fails to learn the underlying patterns.
- A **well-fitted model** performs well, achieving a low training error as it accurately learns the patterns without overcomplicating.
- An **overfit model** performs exceptionally well in training, leading to a very low training error, as it memorizes the data rather than generalizing.

Performance in the Testing Set:

- An **underfit model** continues to perform poorly, resulting in a high testing error due to its inability to generalize or capture patterns effectively.
- A **well-fitted model** performs well, yielding a low testing error by generalizing effectively to unseen data.
- An **overfit model** performs poorly, showing a high testing error as it fails to generalize and instead relies on memorized training data patterns.

OVERCOME OVERFITTING - UNDERFITTING

To ensure our model performs well and generalizes effectively, we introduce an additional step: splitting the dataset into three distinct subsets. This approach allows us to make informed decisions throughout the modeling process. The dataset is divided as follows:

- **Training set:** Used to train the models.
- **Validation set:** Used to evaluate model performance during development and guide model selection.
- **Testing set:** Used only at the end to assess the final model's performance on unseen data.

In practice, the validation set helps us identify the model that balances complexity and generalization. For instance, observing the validation error can guide us to select the most suitable model—such as Model 2 in our example. The testing set is reserved for the final evaluation of the chosen model. If the performance on the testing set is unsatisfactory, we may need to revisit the entire process, including feature engineering, model selection, or dataset preparation.

Typical Dataset Splits: It is common to use the following splits:

- **60-20-20 split:** 60% of the data for training, 20% for validation, and 20% for testing.
- **80-10-10 split:** 80% of the data for training, 10% for validation, and 10% for testing.

These proportions are not fixed rules but practical guidelines. They aim to allocate most of the data for training while keeping sufficient data for validation and testing. This balance ensures that the model is trained on a substantial portion of the dataset while being evaluated rigorously on unseen examples.

REGULARIZATION

Understanding Regularization

Regularization builds on the concept we explored in the section *"An Example of Underfitting and Overfitting Using Polynomial Regression."* There, we observed that simpler models often underfit the data, while more complex models tend to overfit. Previously, our approach involved testing multiple models and selecting the one that achieved the best balance between performance and complexity.

In contrast, regularization eliminates the need to train multiple models. Instead, we train a single model, but during the training process, we simultaneously focus on two objectives: improving the model's predictive performance and reducing its complexity. The key to achieving this balance lies in measuring both performance and complexity together and incorporating this dual focus into the training process.

Understanding Regression Error and Regularization

Regression Error: This represents the quality of the model's predictions, commonly measured using metrics like absolute errors or squared errors, as discussed in Chapter 3.

Regularization Term: This quantifies the complexity of the model, helping to prevent overfitting. Regularization can be defined using either the **L1 norm** or the **L2 norm**, which measure different aspects of model complexity.

To achieve a balance between accuracy and simplicity, we minimize a **modified error** that combines these two components:

$$\text{Total Error} = \text{Regression Error} + \text{Regularization Term}$$

REGULARIZATION

Regularization is a foundational concept in machine learning, and different models are named based on the type of norm used:

1. **Lasso Regression (L1 Regularization):**

When the L1 norm is used as the regularization term, the model is called **Lasso Regression**. Lasso stands for "Least Absolute Shrinkage and Selection Operator." The error function for lasso regression is:

$$\text{Lasso Error} = \text{Regression Error} + \text{L1 Norm}$$

2. L1 regularization promotes sparsity in the model coefficients, effectively performing feature selection.

3. **Ridge Regression (L2 Regularization):**

When the L2 norm is used, the model is referred to as **Ridge Regression**. The name originates from the shape of the error function, where the inclusion of the L2 norm creates a smoother, more "valley-like" surface compared to sharp corners. The error function for ridge regression is:

$$\text{Ridge Error} = \text{Regression Error} + \text{L2 Norm}$$

4. L2 regularization penalizes large coefficients, encouraging the model to distribute importance across features and reduce sensitivity to individual variables.